Explain Capstone Project

https://chatgpt.com/share/6aea417d-af40-83ed-b142-5f629f0eb2df

Date: 2026-11-03 · 30 messages

User

This is my manual to do my capstone project explain me what is the project about and tell me all the references that I should infer from this to start my project with some clear undersating and objective and don't elobrate to big make me understand

ChatGPT

Project 4 – FMCG Inventory Forecasting & Distribution Optimization Analytics

I've gone through your complete 7-page capstone manual. Let me explain it in a simple way, as if we're starting this project from scratch.

First, forget the technical tools for a moment. Understand the **business problem**, then understand what you need to build and why.

1. What is this project actually about?

Imagine a company like D-Mart, Reliance Retail or a supermarket chain.

The company sells thousands of products across multiple stores. It also has warehouses that supply products to those stores.

Now imagine these situations:

- A store sells 100 units of milk daily, but only has 20 units left.
- Another store has 500 units of a product but sells only 5 units daily.
- A warehouse receives a request for 100 units but delivers only 60.
- A replenishment order is delayed.
- A product is selling faster than usual, and the store may run out of stock.

The company needs a system to identify these problems and make better inventory decisions.

That's exactly what your project is about.

<AsyncImageGroup query={["FMCG supermarket grocery shelves retail store","large FMCG distribution warehouse inventory pallets","retail supply chain warehouse delivery trucks"]} aspectRatio="5:4" layout="bento"/>

In simple terms:

Your project will combine sales, inventory, products, stores and warehouse data to help the business understand what is selling, what is running out, what is overstocked and what needs replenishment.

2. What is the main objective?

Your project has six major objectives.

| Objective | What you need to achieve |

|---|---|
Data integration	Combine data from 7 different files.
Data quality	Identify missing, duplicate and incorrect records.
Data warehousing	Build Bronze, Silver and Gold layers in Snowflake.
Business analytics	Calculate sales, inventory and delivery KPIs.
Risk identification	Identify products with high stock-out/reorder risk.
Visualization	Build dashboards using Streamlit.

The final goal is to give management a clear picture of inventory and distribution performance.

3. Understand your 7 source files

This is probably the most important part before starting development.

Your project has **7 source files containing 6,160 records**.

Think of each file as one piece of the business.

<box border radius="lg" gap={3} padding={3}>
<row align="center" justify="between">
1. products.csv
<badge color="info">800 records</badge>
</row>
<AsyncImage query="FMCG packaged grocery products assortment supermarket" aspectRatio="16:5" width="100%"/>
Contains product information:

- Product ID, name, category, brand
- Unit cost and list price
- Shelf life and pack size

Business question: What products does the company sell?
<divider color="subtle"/>
<row align="center" justify="between">
2. stores.csv
<badge color="info">200 records</badge>
</row>
<AsyncImage query="modern supermarket retail storefront" aspectRatio="16:5" width="100%"/>
Contains store information:

- Store ID, name, city, state, region
- Store format and storage capacity

Business question: Where are the products being sold?
<divider color="subtle"/>
<row align="center" justify="between">
3. warehouses.csv
<badge color="info">60 records</badge>
</row>

<AsyncImage query="FMCG distribution warehouse storage racks" aspectRatio="16:5" width="100%"/>

Contains warehouse information:

- Warehouse ID, name and location
- Warehouse type and capacity
- Temperature control

Business question: Where are products stored and distributed from?

<divider color="subtle"/>

<row align="center" justify="between">

4. sales_orders.csv

<badge color="info">1,800 records</badge>

</row>

<AsyncImage query="supermarket cashier retail sales checkout" aspectRatio="16:5" width="100%"/>

Contains sales transactions:

- Order ID, store ID, product ID
- Order date and quantity sold
- Price, discount, tax and net amount

Business question: How much are we selling?

<divider color="subtle"/>

<row align="center" justify="between">

5. inventory_snapshots.csv

<badge color="info">1,500 records</badge>

</row>

<AsyncImage query="warehouse inventory stock counting boxes shelves" aspectRatio="16:5" width="100%"/>

Contains stock information at a particular point in time:

- Product ID and location ID
- On-hand, reserved and available quantity
- Reorder point and safety stock

Business question: How much stock is currently available?

<divider color="subtle"/>

<row align="center" justify="between">

6. replenishment_orders.csv

<badge color="info">1,000 records</badge>

</row>

<AsyncImage query="warehouse loading delivery goods truck distribution" aspectRatio="16:5" width="100%"/>

Contains stock transfer/replenishment information:

- Source warehouse and destination store
- Requested, allocated and dispatched quantities
- Expected and actual delivery dates

Business question: Are warehouses delivering the required stock on time?

<divider color="subtle"/>
<row align="center" justify="between">
7. sales_events.json
<badge color="info">800 records</badge>
</row>
<AsyncImage query="retail point of sale barcode scanning shopping checkout" aspectRatio="16:5" width="100%"/>
Contains sales-event information:

- Event ID, store ID and product ID
- Event timestamp and quantity
- Selling price and promotion information
- Weather and event context

Business question: What additional factors influence product demand?
</box>

Remember these relationships:

- Products connect with sales, inventory and replenishment.
- Stores connect with sales, inventory and replenishment.
- Warehouses connect with replenishment.
- Sales events connect with products and stores.

4. What architecture are you going to build?

Your project follows the **Medallion Architecture (Bronze → Silver → Gold)**.

<box border radius="lg" padding={3} gap={3}>
<box background="#64748b18" radius="md" padding={3} align="center" gap={1}>
<title size="lg">SOURCE DATA</title>
7 CSV and JSON files
<text color="secondary" size="xs">6,160 total records</text>
</box>
<box align="center">
<icon name="arrow-down" color="secondary" size="xl"/>
IDMC – Data Profiling & Cleansing
<text color="secondary" size="xs">Clean • Warning • Reject • Duplicate</text>
<icon name="arrow-down" color="secondary" size="xl"/>
</box>
<box background="#3b82f618" radius="md" padding={3} align="center" gap={1}>
<badge color="info">BRONZE</badge>
Raw Data Layer
<text color="secondary" size="xs">Store source data with metadata and history</text>
</box>
<box align="center">
<icon name="arrow-down" color="secondary" size="xl"/>
</box>

<box background="#d69e3618" radius="md" padding={3} align="center" gap={1}>
<badge color="warning">SILVER</badge>
Cleaned & Validated Data
<text color="secondary" size="xs">Correct data types • Deduplication • Relationship validation</text>
</box>
<box align="center">
<icon name="arrow-down" color="secondary" size="xl"/>
</box>
<box background="#16a34a18" radius="md" padding={3} align="center" gap={1}>
<badge color="success">GOLD</badge>
Dimensions + Facts + Analytical Marts
<text color="secondary" size="xs">Business-ready analytical data</text>
</box>
<box align="center">
<icon name="arrow-down" color="secondary" size="xl"/>
</box>
<grid columns={2} gap={2}>
<grid-item>
<box background="surface-secondary" radius="md" padding={2} align="center">
Snowpark Python
<text color="secondary" size="xs">Reorder risk scoring</text>
</box>
</grid-item>
<grid-item>
<box background="surface-secondary" radius="md" padding={2} align="center">
Streamlit
<text color="secondary" size="xs">Business dashboards</text>
</box>
</grid-item>
</grid>
</box>

Understand each layer:

- **Bronze:** Preserve the original data.
- **Silver:** Make the data clean, standardized and reliable.
- **Gold:** Organize data for business reporting and analysis.

IDMC handles the initial profiling and cleansing. Snowflake handles storage, transformations, warehousing and analytics.

5. What exactly will you build in Snowflake?

Your manual specifies three types of objects.

A. Dimensions – Descriptive information

Table	Purpose
DIM_PRODUCT	Product details
DIM_STORE	Store details
DIM_WAREHOUSE	Warehouse details
DIM_DATE	Calendar information

B. Fact tables – Business transactions and measurements

Table	Purpose
FACT*SALES*ORDER	Sales transactions
FACT*INVENTORY*SNAPSHOT	Stock information
FACT*REPLENISHMENT*ORDER	Replenishment transactions
FACT*SALES*EVENT	Sales events

C. Analytical marts – Final business reporting

Mart	Purpose
MART*SALES*PERFORMANCE	Sales analysis
MART*INVENTORY*HEALTH	Stock health analysis
MART*REPLENISHMENT*PERFORMANCE	Delivery analysis
MART*DEMAND*REORDER_RISK	Demand and reorder risk

The important difference:

Dimensions describe the business, facts store measurable business events, and marts make the data easier to analyze.

6. What business KPIs should you understand?

These KPIs are the actual reason you are building the project.

KPI	Meaning
Net Sales	Total sales revenue after applicable adjustments
Units Sold	Total quantity sold
Average Selling Price	Average price per unit sold
Sell-through Proxy	Sales compared with available stock/supply
Available Stock	On-hand stock minus reserved stock
Stock-out Rate	Percentage of records/locations experiencing stock-outs
Excess Inventory Rate	Percentage of inventory considered excessive
Replenishment Fill Rate	Quantity delivered compared with quantity requested
Delivery Lead Time	Time taken to deliver products
On-time Delivery Rate	Percentage delivered by expected date
Reorder Risk	Products and locations likely to need replenishment

For example:

If a store requests 100 units and receives 80 units:

\[
Fill\ Rate = \frac{80}{100}\times100=80\%
\]

This tells management that the warehouse fulfilled only 80% of the requested quantity.

7. Important technical requirements you must not miss

Your manual has some additional requirements beyond ordinary ETL.

Requirement	What you need to demonstrate
Data quality	Nulls, duplicates, anomalies, outliers and date formats
Incremental loading	Stream and Task for sales_orders
Idempotency	Replaying the same data must not create duplicates
Security	Three roles and masking policies
Snowpark	Low/Medium/High reorder-risk classification
Recovery	Time Travel or Clone
Streamlit	Interactive dashboards
Testing	Source-to-target reconciliation

One important point: Advanced machine learning is NOT required. Your risk score should be explainable using business metrics.

8. Your five-day project plan

This is the order in which your team should work.

<list connector="solid" marker="number" gap={3}>
<list-item>
Day 1 – Understand and design

- Understand all seven datasets.
- Perform data profiling.
- Identify primary keys and relationships.
- Define fact-table grain.
- Design architecture and KPI catalogue.

<badge color="info">Your first priority</badge>
</list-item>
<list-item>
Day 2 – IDMC and Bronze

- Build cleansing and validation mappings.
- Route clean, warning, duplicate and rejected records.
- Create Snowflake stages and file formats.
- Load Bronze tables.

</list-item>
<list-item>
Day 3 – Silver and Gold

- Create dimensions and fact tables.
- Perform transformations.
- Build four analytical marts.
- Validate KPIs and record counts.

</list-item>
<list-item>

Day 4 – Advanced Snowflake

- Implement Stream and Task.
- Test incremental loads and replay.
- Implement RBAC and masking.
- Develop Snowpark risk scoring.
- Test recovery.

</list-item>
<list-item>

Day 5 – Final delivery

- Build Streamlit dashboards.
- Perform end-to-end testing.
- Complete UAT.
- Prepare runbook and presentation.

</list-item>
</list>

---

9. Before starting Day 1, understand these references

These are the concepts you should refer to while designing your project. Don't start coding everything immediately.

Reference	What you should learn
FMCG business process	Product → Store → Sales → Inventory → Replenishment
Data profiling	Nulls, duplicates, outliers, invalid values
Data quality routing	Clean, Warning, Reject, Duplicate
Primary and foreign keys	How seven datasets are connected
Fact-table grain	What one row represents in each fact table
Medallion architecture	Bronze, Silver, Gold
Star schema	Dimensions and facts
Incremental ETL	Stream, Task and idempotency
Snowflake security	RBAC and masking policies
Snowpark Python	Business-based risk scoring
Streamlit	Dashboard creation

10. What I recommend you do FIRST

Before opening IDMC or writing Snowflake SQL, complete these five things.

{@body const tasks = ["Understand the purpose of all 7 source files.","Identify the primary key of each file.","Identify how the 7 files are related.","Profile all datasets and document data-quality issues.","Prepare a source-to-target mapping and KPI catalogue."]}

{@body const [done,setDone] = DIL.useState([])}

<box border radius="lg" padding={3} gap={3}>

<row justify="between" align="center">

Day 1 preparation checklist

<badge color={done.length===5?"success":"info"}>{done.length}/5 completed</badge>

</row>

<box background="surface-tertiary" radius="full" height="7px" clip>

<box background="#16a34a" height="7px" radius="full" width={`${done.length/5*100}%`}/>

</box>

{#each tasks as task,i}

<checkbox key={i} checked={done.includes(i)} onChange={v=>setDone(old=>v?[...old,i]:old.filter(x=>x!==i))}>{task}</checkbox>

{/each}

</box>

My advice: Don't think of this as just an IDMC or Snowflake project. Think of yourself as building a complete data platform for an FMCG company.

Every technical component has a business reason:

- IDMC makes incoming data trustworthy.
- Snowflake stores and organizes the data.
- Snowpark identifies inventory risk.
- Streamlit helps management understand the results.

One question to keep in mind throughout the project:

"Can our system help a retail company understand what products to sell, where stock is insufficient, where excess stock exists, and whether replenishment is happening on time?"

If you can answer that question using your final dashboard, you've achieved the main objective of this capstone.

Source: Your uploaded Project 4 – Updated Snowflake Capstone document, sections 1–17.

User

{"content*type*:"*image*asset*pointer*","*asset*pointer":"sediment://file_*00000000be1c8246b8e70bf83ddcb9cc?shared*conversation_*id=6aea417d-af40-83ed-b142-5f629f0eb2df*","*size*bytes":14064,"width":963,"height":531,"metadata":{"sanitized":true}}

{"content*type*:"*image*asset*pointer*","*asset*pointer":"sediment://file_*00000000053481f4956c7003aaf3bc90?shared*conversation_*id=6aea417d-af40-83ed-b142-5f629f0eb2df*","*size*bytes":6492,"width":1248,"height":105,"metadata":{"sanitized":true}}

{"content*type*:"*image*asset*pointer*","*asset*pointer":"sediment://file_*000000008f14820a82df94325b93350a?shared*conversation

id=6aea417d-af40-83ed-b142-5f629f0eb2df","*size*
bytes":32884,"width":1358,"height":578,"metadata":{"sanitized":true}}
Ok now I have some under standing on the project now my part to deal with the replenishment*order, so far I have did this and for the o*qty_error in the expression I have used this logic IIF(
(
ISNULL(requested_qty)
OR ISNULL(allocated_qty)
OR ISNULL(dispatched_qty)
OR ISNULL(delivered_qty)
OR requested_qty < 0
OR allocated_qty < 0
OR dispatched_qty < 0
OR delivered_qty < 0
),
'INVALID',
'VALID'
)

ChatGPT

Good. You're now working on the **Replenishment Orders transformation**, and your approach is correct so far. Let's review what you've implemented and what you should improve before moving to the Router.

Your responsibility is to make sure replenishment records are valid before they enter the clean dataset.

1. What you have implemented so far

From your screenshots, you've created these expressions:

Output field	Purpose
O*REP*ID	Trim replenishment ID
O*WAR*ID	Trim warehouse ID
O*DES*ID	Trim destination store ID
O*PRO*ID	Trim product ID
V*RES*DATE	Convert request date
V*ALLO*DATE	Convert allocation date
V*DIS*DATE	Convert dispatch date
V*DEL*DATE	Convert delivered date
O*RES*DATE_ERROR	Validate request date
O*ALLO*DATE_ERROR	Validate allocation date
O*DIS*DATE_ERROR	Validate dispatch date
O*DEL*DATE_ERROR	Validate delivered date
DATE*SEQ*ERROR	Validate date sequence

O*QTY*ERROR	Validate quantity fields
O*REP*STATUS	Standardize replenishment status
O*PRIORITY*CODE	Standardize priority code

This is a good structure because you're separating transformation, validation and routing.

2. Review of your OQTYERROR logic

Your current expression:

```
IIF(
    (
        ISNULL(requested_qty)
        OR ISNULL(allocated_qty)
        OR ISNULL(dispatched_qty)
        OR ISNULL(delivered_qty)

        OR requested_qty < 0
        OR allocated_qty < 0
        OR dispatched_qty < 0
        OR delivered_qty < 0
    ),
    'INVALID',
    'VALID'
)
```

Your business logic is correct, assuming all four quantities are mandatory and negative quantities are not allowed.

However, there is one important issue.

Your manual defines these source columns as VARCHAR:

- requested_qty
- allocated_qty
- dispatched_qty
- delivered_qty

You should validate whether these fields contain numeric values before comparing them with zero. Otherwise, non-numeric strings could cause expression errors.

Good. You're now working on the **Replenishment Orders transformation**, and your approach is correct so far. Let's review what you've implemented and what you should improve before moving to the Router.

Your responsibility is to make sure replenishment records are valid before they enter the clean dataset.

1. What you have implemented so far

From your screenshots, you've created these expressions:

Output field	Purpose
O*REP*ID	Trim replenishment ID
O*WAR*ID	Trim warehouse ID

O*DES*ID	Trim destination store ID
O*PRO*ID	Trim product ID
V*RES*DATE	Convert request date
V*ALLO*DATE	Convert allocation date
V*DIS*DATE	Convert dispatch date
V*DEL*DATE	Convert delivered date
O*RES*DATE_ERROR	Validate request date
O*ALLO*DATE_ERROR	Validate allocation date
O*DIS*DATE_ERROR	Validate dispatch date
O*DEL*DATE_ERROR	Validate delivered date
DATE*SEQ*ERROR	Validate date sequence
O*QTY*ERROR	Validate quantity fields
O*REP*STATUS	Standardize replenishment status
O*PRIORITY*CODE	Standardize priority code

This is a good structure because you're separating transformation, validation and routing.

2. Review of your OQTYERROR logic

Your current expression:

```
IIF(
    (
        ISNULL(requested_qty)
        OR ISNULL(allocated_qty)
        OR ISNULL(dispatched_qty)
        OR ISNULL(delivered_qty)

        OR requested_qty < 0
        OR allocated_qty < 0
        OR dispatched_qty < 0
        OR delivered_qty < 0
    ),
    'INVALID',
    'VALID'
)
```

Your business logic is correct, assuming all four quantities are mandatory and negative quantities are not allowed.

However, there is one important issue.

Your manual defines these source columns as VARCHAR:

- requested_qty
- allocated_qty
- dispatched_qty
- delivered_qty

You should validate whether these fields contain numeric values before comparing them with zero. Otherwise, non-numeric strings could cause expression errors.

My recommended improvement

Create numeric validation ports first, then use those ports for the final quantity check.

For example, validate each source quantity using:

```
REG_MATCH(TRIM(requested_qty), '^-?[0-9]+$')
```

This checks whether the value contains a valid integer representation.

Apply the same validation to the other three quantity fields.

Then your quantity validation should account for:

1. NULL values.
2. Non-numeric values.
3. Negative values.

If zero quantities are not permitted by your business rules, include zero as invalid too. Your manual does not explicitly specify that rule.

3. Review your DATESEQERROR

Your current expression is:

```
IIF(
    V_DIS_DATE < V_RES_DATE,
    'INVALID',
    'VALID'
)
```

This checks whether dispatch happens before the request date.

But your replenishment process has five dates:

1. Request date
2. Allocation date
3. Dispatch date
4. Expected delivery date
5. Delivered date

I recommend checking these chronological rules:

Validation	Condition
Allocation	Allocation date cannot be before request date
Dispatch	Dispatch cannot be before allocation
Expected delivery	Expected delivery cannot be before dispatch
Actual delivery	Delivered date cannot be before dispatch

A late delivery should not automatically be rejected. It is a business performance issue, not necessarily invalid data.

4. Most important: Correct your Router conditions

Looking at your first screenshot, your VALID group is good.

However, your INVALID group currently checks only NULL dates.

That means a record with a negative quantity or an incorrectly formatted date could potentially fail to enter the correct output group.

I recommend this structure:

VALID output group

```
O_RES_DATE_ERROR = 'VALID'
AND O_ALLO_DATE_ERROR = 'VALID'
AND O_DIS_DATE_ERROR = 'VALID'
AND O_DEL_DATE_ERROR = 'VALID'
AND DATE_SEQ_ERROR = 'VALID'
AND O_QTY_ERROR = 'VALID'
```

INVALID output group

```
O_RES_DATE_ERROR <> 'VALID'
OR O_ALLO_DATE_ERROR <> 'VALID'
OR O_DIS_DATE_ERROR <> 'VALID'
OR O_DEL_DATE_ERROR <> 'VALID'
OR DATE_SEQ_ERROR <> 'VALID'
OR O_QTY_ERROR <> 'VALID'
```

This also catches `UNKNOWN` values.

Make sure your expected delivery date is validated too, preferably with its own expression port.

5. Your final transformation flow

<box border radius="lg" padding={3} gap={2} align="center">
<box background="surface-secondary" padding={3} radius="md" width="100%" align="center">
Source – replenishment_orders.csv
</box>
<icon name="arrow-down" color="secondary" size="lg"/>
<box background="surface-secondary" padding={3} radius="md" width="100%" align="center">
Expression Transformation
<text color="secondary" size="xs">Trim • Date conversion • Quantity validation • Date sequence • Status standardization</text>
</box>
<icon name="arrow-down" color="secondary" size="lg"/>
Router Transformation
<grid columns={2} gap={2}>
<grid-item>
<box background="#16a34a18" radius="md" padding={2} align="center">
<badge color="success">VALID</badge>
Clean records
<text color="secondary" size="xs">All validations passed</text>
</box>
</grid-item>
<grid-item>
<box background="#dc262618" radius="md" padding={2} align="center">

<badge color="danger">INVALID</badge>

Rejected records

<text color="secondary" size="xs">One or more validations failed</text>

</box>

</grid-item>

</grid>

</box>

My suggestion: Before moving forward, complete these three things:

- Add numeric-format validation for all four quantity fields.
- Expand your date-sequence validation.
- Correct the INVALID Router condition.

Once these are done, your replenishment-orders cleansing logic will be much more reliable.

My recommended improvement

Create numeric validation ports first, then use those ports for the final quantity check.

For example, validate each source quantity using:

```
REG_MATCH(TRIM(requested_qty), '^-?[0-9]+$')
```

This checks whether the value contains a valid integer representation.

Apply the same validation to the other three quantity fields.

Then your quantity validation should account for:

1. NULL values.
2. Non-numeric values.
3. Negative values.

If zero quantities are not permitted by your business rules, include zero as invalid too. Your manual does not explicitly specify that rule.

3. Review your DATESEQERROR

Your current expression is:

```
IIF(
    V_DIS_DATE < V_RES_DATE,
    'INVALID',
    'VALID'
)
```

This checks whether dispatch happens before the request date.

But your replenishment process has five dates:

1. Request date
2. Allocation date
3. Dispatch date
4. Expected delivery date
5. Delivered date

I recommend checking these chronological rules:

Validation	Condition
Allocation	Allocation date cannot be before request date
Dispatch	Dispatch cannot be before allocation
Expected delivery	Expected delivery cannot be before dispatch
Actual delivery	Delivered date cannot be before dispatch

A late delivery should not automatically be rejected. It is a business performance issue, not necessarily invalid data.

4. Most important: Correct your Router conditions

Looking at your first screenshot, your VALID group is good.

However, your INVALID group currently checks only NULL dates.

That means a record with a negative quantity or an incorrectly formatted date could potentially fail to enter the correct output group.

I recommend this structure:

VALID output group

```
O_RES_DATE_ERROR = 'VALID'
AND O_ALLO_DATE_ERROR = 'VALID'
AND O_DIS_DATE_ERROR = 'VALID'
AND O_DEL_DATE_ERROR = 'VALID'
AND DATE_SEQ_ERROR = 'VALID'
AND O_QTY_ERROR = 'VALID'
```

INVALID output group

```
O_RES_DATE_ERROR <> 'VALID'
OR O_ALLO_DATE_ERROR <> 'VALID'
OR O_DIS_DATE_ERROR <> 'VALID'
OR O_DEL_DATE_ERROR <> 'VALID'
OR DATE_SEQ_ERROR <> 'VALID'
OR O_QTY_ERROR <> 'VALID'
```

This also catches `UNKNOWN` values.

Make sure your expected delivery date is validated too, preferably with its own expression port.

5. Your final transformation flow

<box border radius="lg" padding={3} gap={2} align="center">
<box background="surface-secondary" padding={3} radius="md" width="100%" align="center">
Source – replenishment_orders.csv
</box>
<icon name="arrow-down" color="secondary" size="lg"/>
<box background="surface-secondary" padding={3} radius="md" width="100%" align="center">

Expression Transformation

<text color="secondary" size="xs">Trim • Date conversion • Quantity validation • Date sequence • Status standardization</text>

</box>

<icon name="arrow-down" color="secondary" size="lg"/>

Router Transformation

<grid columns={2} gap={2}>

<grid-item>

<box background="#16a34a18" radius="md" padding={2} align="center">

<badge color="success">VALID</badge>

Clean records

<text color="secondary" size="xs">All validations passed</text>

</box>

</grid-item>

<grid-item>

<box background="#dc262618" radius="md" padding={2} align="center">

<badge color="danger">INVALID</badge>

Rejected records

<text color="secondary" size="xs">One or more validations failed</text>

</box>

</grid-item>

</grid>

</box>

My suggestion: Before moving forward, complete these three things:

- Add numeric-format validation for all four quantity fields.
- Expand your date-sequence validation.
- Correct the INVALID Router condition.

Once these are done, your replenishment-orders cleansing logic will be much more reliable.

User

ok so you have to understand one thing here in the router i have created two group valid and invalid and there is one default group is there that will automaticaly collect the remainnig rows and I just mapped them to a trarget with by naming it as warning

ChatGPT

Got it. Your Router has **three output paths**, not just two. I understand your design now.

Your structure is:

| Router output | Purpose | Target |

|---|---|---|

| VALID | All mandatory validations pass | Clean target |

| INVALID | Records fail mandatory validations | Reject target |

| DEFAULT | Records that don't match VALID or INVALID conditions | Warning target |

How this works in your project

For example:

Scenario	Output
All dates and quantities are valid	VALID
Request date is NULL	INVALID
Quantity is negative	INVALID
Optional field is NULL	DEFAULT → WARNING
Unusual but acceptable business value	DEFAULT → WARNING

One important point: The DEFAULT group automatically receives records that don't satisfy any explicitly defined Router group condition.

So your design is perfectly reasonable:

- VALID = Good records.
- INVALID = Bad records that must be rejected.
- DEFAULT = Records that need attention but can potentially be retained.

I'll follow this three-way routing design when helping you with the remaining transformations.

User

this i got in cleaned

replenishment*id,source*warehouse*id,destination*store*id,product*id,request*date,allocated*date,dispatch*date,delivered*date,replenishment*status,priority*code,expected*delivery*date,requested*qty,allocated*qty,dispatched*qty,delivered*qty,source*updated*ts
REP0000021,WH028,STR0032,PRD000274,02/04/2025 00:00:00.000000,02/05/2025 00:00:00.000000,02/02/2025 00:00:00.000000,02/09/2025 00:00:00.000000,DELIVERED,URGENT,2025-02-08,795,780,777,788,2025-02-09T00:00:00Z

this in warnning

expected*delivery*date,requested*qty,allocated*qty,dispatched*qty,delivered*qty,source*updated*ts,replenishment*id,source*warehouse*id,destination*store*id,product*id,request*date,allocated*date,dispatch*date,delivered*date,replenishment*status,priority*code
2025-10-18,126,111,104,96,2025-10-19T00:00:00Z,REP0000071,WH018,STR0182,PRD000125,10/12/2 00:00:00.000000,10/13/2025 00:00:00.000000,10/14/2025 00:00:00.000000,,DELIVERED,CRITICAL

this in error log

ChatGPT

import pandas as pd
p='/mnt/data/cleaned*replishment*orders.csv'
df=pd.read_csv(p)
df.shape, df.columns.tolist(), df.head(10).to_dict('records')

Tool

((998, 16),
['replenishment_id',
'source*warehouse*id',
'destination*store*id',
'product_id',
'request_date',
'allocated_date',
'dispatch_date',
'expected*delivery*date',
'delivered_date',
'replenishment_status',
'priority_code',
'requested_qty',
'allocated_qty',
'dispatched_qty',
'delivered_qty',
'source*updated*ts'],
[{'replenishment_id': 'REP0000001',
'source*warehouse*id': 'WH008',
'destination*store*id': 'STR0012',
'product_id': 'PRD000014',
'request_date': '03/17/2025 00:00:00.000000',
'allocated_date': '03/18/2025 00:00:00.000000',
'dispatch_date': '03/19/2025 00:00:00.000000',
'expected*delivery*date': '2025-03-21',
'delivered_date': '03/20/2025 00:00:00.000000',
'replenishment_status': 'DELIVERED',
'priority_code': 'NORMAL',
'requested_qty': 276,
'allocated_qty': 264,
'dispatched_qty': 256,
'delivered_qty': 265,
'source*updated*ts': '2025-03-22T00:00:00Z'},
{'replenishment_id': 'REP0000002',
'source*warehouse*id': 'WH015',
'destination*store*id': 'STR0023',
'product_id': 'PRD000027',
'request_date': '02/27/2025 00:00:00.000000',
'allocated_date': '02/28/2025 00:00:00.000000',
'dispatch_date': '03/01/2025 00:00:00.000000',
'expected*delivery*date': '2025-03-06',
'delivered_date': '03/06/2025 00:00:00.000000',
'replenishment_status': 'DELIVERED',

'priority_code': 'NORMAL',
'requested_qty': 617,
'allocated_qty': 606,
'dispatched_qty': 601,
'delivered_qty': 597,
'source*updated*ts': '2025-03-07T00:00:00Z'},
{'replenishment_id': 'REP0000003',
'source*warehouse*id': 'WH022',
'destination*store*id': 'STR0034',
'product_id': 'PRD000040',
'request_date': '04/04/2025 00:00:00.000000',
'allocated_date': '04/05/2025 00:00:00.000000',
'dispatch_date': '04/06/2025 00:00:00.000000',
'expected*delivery*date': '2025-04-10',
'delivered_date': '04/11/2025 00:00:00.000000',
'replenishment_status': 'DELIVERED',
'priority_code': 'URGENT',
'requested_qty': 633,
'allocated_qty': 619,
'dispatched_qty': 609,
'delivered_qty': 631,
'source*updated*ts': '2025-04-11T00:00:00Z'},
{'replenishment_id': 'REP0000004',
'source*warehouse*id': 'WH029',
'destination*store*id': 'STR0045',
'product_id': 'PRD000053',
'request_date': '11/01/2025 00:00:00.000000',
'allocated_date': '11/02/2025 00:00:00.000000',
'dispatch_date': '11/03/2025 00:00:00.000000',
'expected*delivery*date': '2025-11-06',
'delivered_date': '11/06/2025 00:00:00.000000',
'replenishment_status': 'DELIVERED',
'priority_code': 'NORMAL',
'requested_qty': 695,
'allocated_qty': 693,
'dispatched_qty': 687,
'delivered_qty': 671,
'source*updated*ts': '2025-11-07T00:00:00Z'},
{'replenishment_id': 'REP0000005',
'source*warehouse*id': 'WH036',
'destination*store*id': 'STR0056',
'product_id': 'PRD000066',
'request_date': '02/21/2025 00:00:00.000000',

'allocated_date': '02/22/2025 00:00:00.000000',
'dispatch_date': '02/23/2025 00:00:00.000000',
'expected*delivery*date': '2025-02-27',
'delivered_date': '02/28/2025 00:00:00.000000',
'replenishment_status': 'DELIVERED',
'priority_code': 'URGENT',
'requested_qty': 688,
'allocated_qty': 671,
'dispatched_qty': 665,
'delivered_qty': 664,
'source*updated*ts': '2025-02-28T00:00:00Z'},
{'replenishment_id': 'REP0000006',
'source*warehouse*id': 'WH043',
'destination*store*id': 'STR0067',
'product_id': 'PRD000079',
'request_date': '02/20/2025 00:00:00.000000',
'allocated_date': '02/21/2025 00:00:00.000000',
'dispatch_date': '02/22/2025 00:00:00.000000',
'expected*delivery*date': '2025-02-25',
'delivered_date': '12/31/9999 00:00:00.000000',
'replenishment_status': 'DISPATCHED',
'priority_code': 'URGENT',
'requested_qty': 395,
'allocated_qty': 392,
'dispatched_qty': 375,
'delivered_qty': 0,
'source*updated*ts': '2025-02-26T00:00:00Z'},
{'replenishment_id': 'REP0000007',
'source*warehouse*id': 'WH050',
'destination*store*id': 'STR0078',
'product_id': 'PRD000092',
'request_date': '10/06/2025 00:00:00.000000',
'allocated_date': '10/07/2025 00:00:00.000000',
'dispatch_date': '10/08/2025 00:00:00.000000',
'expected*delivery*date': '2025-10-12',
'delivered_date': '10/13/2025 00:00:00.000000',
'replenishment_status': 'DELIVERED',
'priority_code': 'CRITICAL',
'requested_qty': 67,
'allocated_qty': 49,
'dispatched_qty': 51,
'delivered_qty': 51,
'source*updated*ts': '2025-10-13T00:00:00Z'},

{'replenishment_id': 'REP0000008',
'source*warehouse*id': 'WH057',
'destination*store*id': 'STR0089',
'product_id': 'PRD000105',
'request_date': '07/11/2025 00:00:00.000000',
'allocated_date': '07/12/2025 00:00:00.000000',
'dispatch_date': '07/13/2025 00:00:00.000000',
'expected*delivery*date': '2025-07-15',
'delivered_date': '07/15/2025 00:00:00.000000',
'replenishment_status': 'DELIVERED',
'priority_code': 'URGENT',
'requested_qty': 430,
'allocated_qty': 428,
'dispatched_qty': 416,
'delivered_qty': 406,
'source*updated*ts': '2025-07-16T00:00:00Z'},
{'replenishment_id': 'REP0000009',
'source*warehouse*id': 'WH004',
'destination*store*id': 'STR0100',
'product_id': 'PRD000118',
'request_date': '05/08/2025 00:00:00.000000',
'allocated_date': '05/09/2025 00:00:00.000000',
'dispatch_date': '05/10/2025 00:00:00.000000',
'expected*delivery*date': '2025-05-15',
'delivered_date': '05/16/2025 00:00:00.000000',
'replenishment_status': 'DELIVERED',
'priority_code': 'URGENT',
'requested_qty': 561,
'allocated_qty': 561,
'dispatched_qty': 556,
'delivered_qty': 527,
'source*updated*ts': '2025-05-16T00:00:00Z'},
{'replenishment_id': 'REP0000010',
'source*warehouse*id': 'WH011',
'destination*store*id': 'STR0111',
'product_id': 'PRD000131',
'request_date': '11/01/2025 00:00:00.000000',
'allocated_date': '11/02/2025 00:00:00.000000',
'dispatch_date': '11/03/2025 00:00:00.000000',
'expected*delivery*date': '2025-11-06',
'delivered_date': '11/06/2025 00:00:00.000000',
'replenishment_status': 'DELIVERED',
'priority_code': 'NORMAL',

'requested_qty': 613,
'allocated_qty': 607,
'dispatched_qty': 601,
'delivered_qty': 613,
'source*updated*ts': '2025-11-07T00:00:00Z'}])

ChatGPT

for rid in ['REP0000021','REP0000071']:
print(rid)
display(df[df.replenishment_id==rid])

Tool

REP0000021
REP0000071

ChatGPT

df[df.astype(str).apply(lambda c:c.str.contains('REP0000021|REP0000071', regex=True)).any(axis=1)]

Tool

Empty DataFrame
Columns: [replenishment*id, source*warehouse*id, destination*store*id, product*id, request *date, allocated*date, dispatch*date, expected*delivery*date, delivered*date, replenishment *status, priority*code, requested*qty, allocated*qty, dispatched*qty, delivered*qty, source *updated*ts]
Index: []

ChatGPT

Now we have actual output from your IDMC mapping. This is where we need to check whether your **Clean, Warning and Error routing is working according to the business rules**, not just whether the mapping runs successfully.

I can already identify an important issue in your Clean output.

1. Clean output – There is a data-quality problem

Your record:

Column	Value
replenishment_id	REP0000021
request_date	02/04/2025
dispatch_date	02/02/2025
delivered_date	02/09/2025
expected*delivery*date	2025-02-08
requested_qty	795
allocated_qty	780
dispatched_qty	777

| delivered_qty | 788 |

Problem 1: Date sequence

Your dispatch date is February 2, but your request date is February 4.

That means the warehouse dispatched the goods before the store requested them.

According to your project manual, this is a business anomaly that must be rejected.

Your `DATE_SEQ_ERROR` should identify this record as `INVALID`.

Problem 2: Quantity sequence

Your delivered quantity is 788, but dispatched quantity is only 777.

The company cannot normally deliver more units than it dispatched.

This should be identified as a business anomaly. Your current `O_QTY_ERROR` doesn't check this relationship.

Problem 3: Late delivery

Expected delivery: February 8.

Actual delivery: February 9.

This is a late delivery, but it should not automatically be rejected. It is a delivery-performance issue.

---

2. Warning output – This needs attention

Your record:

Column	Value
replenishment_id	REP0000071
request_date	10/12/2025
allocation_date	10/13/2025
dispatch_date	10/14/2025
delivered_date	NULL
expected*delivery*date	2025-10-18
replenishment_status	DELIVERED
priority_code	CRITICAL

Here the dates are sequential, but there is a problem:

The replenishment status says `DELIVERED`, while the delivered date is NULL.

This is a business inconsistency.

I agree with routing this record to WARNING rather than automatically rejecting it, unless your business rules explicitly make delivered date mandatory for delivered orders.

Now we have actual output from your IDMC mapping. This is where we need to check whether your **Clean, Warning and Error routing is working according to the business rules**, not just whether the mapping runs successfully.

I can already identify an important issue in your Clean output.

1. Clean output – There is a data-quality problem

Your record:

Column	Value
replenishment_id	REP0000021
request_date	02/04/2025
dispatch_date	02/02/2025
delivered_date	02/09/2025
expected*delivery*date	2025-02-08
requested_qty	795
allocated_qty	780
dispatched_qty	777
delivered_qty	788

Problem 1: Date sequence

Your dispatch date is February 2, but your request date is February 4.

That means the warehouse dispatched the goods before the store requested them.

According to your project manual, this is a business anomaly that must be rejected.

Your `DATE_SEQ_ERROR` should identify this record as `INVALID`.

Problem 2: Quantity sequence

Your delivered quantity is 788, but dispatched quantity is only 777.

The company cannot normally deliver more units than it dispatched.

This should be identified as a business anomaly. Your current `O_QTY_ERROR` doesn't check this relationship.

Problem 3: Late delivery

Expected delivery: February 8.

Actual delivery: February 9.

This is a late delivery, but it should not automatically be rejected. It is a delivery-performance issue.

---

2. Warning output – This needs attention

Your record:

Column	Value
replenishment_id	REP0000071
request_date	10/12/2025
allocation_date	10/13/2025
dispatch_date	10/14/2025

delivered_date	NULL
expected*delivery*date	2025-10-18
replenishment_status	DELIVERED
priority_code	CRITICAL

Here the dates are sequential, but there is a problem:

The replenishment status says `DELIVERED`, while the delivered date is NULL.

This is a business inconsistency.

I agree with routing this record to WARNING rather than automatically rejecting it, unless your business rules explicitly make delivered date mandatory for delivered orders.

3. One important correction in your date conversion

Looking at your earlier screenshot, you used:

```
TO_DATE(request_date, 'YYYY-MM-DD')
```

But your actual source data looks like:

```
02/04/2025 00:00:00.000000
```

These formats do not match.

For request, allocation, dispatch and delivered dates, use the appropriate source format:

```
TO_DATE(
    request_date,
    'MM/DD/YYYY HH24:MI:SS.FF'
)
```

Apply the same format to the other three date fields.

Your expected delivery date has a different format:

```
2025-02-08
```

For that field, use:

```
TO_DATE(expected_delivery_date, 'YYYY-MM-DD')
```

This is important because incorrect date parsing can affect your validation and routing.

4. What I recommend adding to your validations

Your current quantity validation checks NULL and negative values. Keep that, but add business relationship checks.

Additional validation	Expected handling
Delivered quantity greater than dispatched	Warning or reject based on business rule
Dispatched quantity greater than allocated	Warning or reject
Allocated quantity greater than requested	Warning
Delivered status but delivered date is NULL	Warning
Delivered date before dispatch date	Reject
Dispatch date before request date	Reject
Actual delivery after expected delivery	Warning

These are additional recommended business rules, not all mandatory anomalies specified in your manual.

5. My assessment of your current output

<box border radius="lg" padding={3} gap={3}>
<row align="center" justify="between">
Clean output
<badge color="danger">Needs correction</badge>
</row>
REP0000021 should not be clean because dispatch occurred before the request date.
<divider color="subtle"/>
<row align="center" justify="between">
Warning output
<badge color="warning">Reasonable routing</badge>
</row>
REP0000071 has a missing delivered date despite having DELIVERED status.
<divider color="subtle"/>
<row align="center" justify="between">
Error log
<badge color="info">Needs inspection</badge>
</row>
I haven't received the actual error-log file or its row details yet, so I cannot verify those errors.
</box>

My recommendation: Don't change your entire mapping. First correct the date conversion, then test `DATE_SEQ_ERROR` and add the quantity relationship checks.

One thing I want you to follow throughout this project: distinguish between a technically invalid record and a business warning. A late delivery is not the same as an impossible delivery sequence.

That distinction will make your IDMC data-quality implementation much stronger.

3. One important correction in your date conversion

Looking at your earlier screenshot, you used:

```
TO_DATE(request_date, 'YYYY-MM-DD')
```

But your actual source data looks like:

```
02/04/2025 00:00:00.000000
```

These formats do not match.

For request, allocation, dispatch and delivered dates, use the appropriate source format:

```
TO_DATE(
    request_date,
    'MM/DD/YYYY HH24:MI:SS.FF'
)
```

Apply the same format to the other three date fields.

Your expected delivery date has a different format:

```
2025-02-08
```

For that field, use:

```
TO_DATE(expected_delivery_date, 'YYYY-MM-DD')
```

This is important because incorrect date parsing can affect your validation and routing.

4. What I recommend adding to your validations

Your current quantity validation checks NULL and negative values. Keep that, but add business relationship checks.

Additional validation	Expected handling
Delivered quantity greater than dispatched	Warning or reject based on business rule
Dispatched quantity greater than allocated	Warning or reject
Allocated quantity greater than requested	Warning
Delivered status but delivered date is NULL	Warning
Delivered date before dispatch date	Reject
Dispatch date before request date	Reject
Actual delivery after expected delivery	Warning

These are additional recommended business rules, not all mandatory anomalies specified in your manual.

5. My assessment of your current output

<box border radius="lg" padding={3} gap={3}>
<row align="center" justify="between">
Clean output
<badge color="danger">Needs correction</badge>
</row>
REP0000021 should not be clean because dispatch occurred before the request date.
<divider color="subtle"/>
<row align="center" justify="between">
Warning output
<badge color="warning">Reasonable routing</badge>
</row>
REP0000071 has a missing delivered date despite having DELIVERED status.
<divider color="subtle"/>
<row align="center" justify="between">
Error log
<badge color="info">Needs inspection</badge>
</row>
I haven't received the actual error-log file or its row details yet, so I cannot verify those errors.

</box>

My recommendation: Don't change your entire mapping. First correct the date conversion, then test `DATE_SEQ_ERROR` and add the quantity relationship checks.

One thing I want you to follow throughout this project: distinguish between a technically invalid record and a business warning. A late delivery is not the same as an impossible delivery sequence.

That distinction will make your IDMC data-quality implementation much stronger.

User

the cleaned*replenishment*orders is the cleaned data the one with date issue is warning and the null value is error is it correct or not

ChatGPT

Not completely correct. Let's compare your outputs with the business rules in your capstone manual.

1. Correct classification

Scenario	Your output	Expected output
Dispatch date before request date	Warning	<badge color="danger">Reject / Error</badge>
Delivered date is NULL	Error	<badge color="warning">Warning</badge> if optional
Negative quantity	Error	<badge color="danger">Reject / Error</badge>
Optional field is NULL	Error	<badge color="warning">Warning</badge>
Correct dates and quantities	Clean	<badge color="success">Clean</badge>

2. Why?

Your project manual specifies these routing rules:

- Valid records → Clean.
- Correctable date formats → Standardize and retain.
- Optional NULL values → Warning.
- Impossible date sequences → Reject.
- Extreme but possible values → Warning and retain.

3. Apply this to your records

Record REP0000021

- Request date: February 4
- Dispatch date: February 2

Dispatch happened before the request. This is an impossible business sequence.

It should go to **ERROR/REJECT**, not WARNING.

Record REP0000071

- Delivered date: NULL
- Status: DELIVERED

This is a business inconsistency. If delivered date is optional in your general schema, route it to WARNING. If your business rule requires a delivery date whenever status is DELIVERED, route it to ERROR.

4. My recommendation

Keep your three Router outputs:

1. VALID → Clean
2. INVALID → Error/Reject
3. DEFAULT → Warning

But don't classify every NULL as an error. Your manual specifically distinguishes optional NULLs from impossible business sequences.

One important question: Is `delivered_date` mandatory in your source-data business rules, or are you treating it as optional? This will determine the correct routing for REP0000071.

User

do then give me condition for reject, warning, clean and duplicates so that I can have four targets

ChatGPT

For your replenishment mapping, I recommend using **four separate Router outputs and four targets**:

1. CLEAN – Valid records.
2. REJECT – Records with mandatory data errors or impossible business conditions.
3. WARNING – Records with optional missing values or business conditions that need attention.
4. DUPLICATE – Repeated replenishment records.

We'll use your existing Expression transformation and add only the necessary validation ports.

1. First, define the business rules

Validation	Classification
All mandatory fields valid	CLEAN
Mandatory date is NULL or invalid	REJECT
Quantity is NULL, non-numeric or negative	REJECT
Dispatch date before request date	REJECT
Optional delivered date is NULL	WARNING
Delivered status but delivered date is NULL	WARNING (assuming optional)
Delivery after expected delivery date	WARNING
Duplicate replenishment ID	DUPLICATE

Your manual specifically identifies dispatch before request as an anomaly that must be rejected. The other additional rules are recommended validation rules.

For your replenishment mapping, I recommend using **four separate Router outputs and four targets**:

1. CLEAN – Valid records.
2. REJECT – Records with mandatory data errors or impossible business conditions.
3. WARNING – Records with optional missing values or business conditions that need attention.
4. DUPLICATE – Repeated replenishment records.

We'll use your existing Expression transformation and add only the necessary validation ports.

1. First, define the business rules

Validation	Classification
All mandatory fields valid	CLEAN
Mandatory date is NULL or invalid	REJECT
Quantity is NULL, non-numeric or negative	REJECT
Dispatch date before request date	REJECT
Optional delivered date is NULL	WARNING
Delivered status but delivered date is NULL	WARNING (assuming optional)
Delivery after expected delivery date	WARNING
Duplicate replenishment ID	DUPLICATE

Your manual specifically identifies dispatch before request as an anomaly that must be rejected. The other additional rules are recommended validation rules.

2. Router conditions

Assume you create an `O_DUPLICATE_FLAG` field containing `Y` or `N`.

<box border radius="lg" padding={3} gap={2}>
<row justify="between" align="center">
Group 1 – DUPLICATE
<badge color="discovery">Duplicate target</badge>
</row>

```
  O_DUPLICATE_FLAG = 'Y'
```

<divider color="subtle"/>
<row justify="between" align="center">
Group 2 – REJECT
<badge color="danger">Error target</badge>
</row>

```
  O_DUPLICATE_FLAG = 'N'
  AND
  (
      O_RES_DATE_ERROR <> 'VALID'
      OR O_ALLO_DATE_ERROR <> 'VALID'
      OR O_DIS_DATE_ERROR <> 'VALID'
      OR O_EXP_DATE_ERROR <> 'VALID'
```

```
      OR DATE_SEQ_ERROR = 'INVALID'
      OR O_QTY_ERROR <> 'VALID'
  )
```

Here, `O_EXP_DATE_ERROR` is a new validation port for expected delivery date. Add it to your mapping.

<divider color="subtle"/>

<row justify="between" align="center">

Group 3 – CLEAN

<badge color="success">Clean target</badge>

</row>

```
  O_DUPLICATE_FLAG = 'N'
  AND O_RES_DATE_ERROR = 'VALID'
  AND O_ALLO_DATE_ERROR = 'VALID'
  AND O_DIS_DATE_ERROR = 'VALID'
  AND O_DEL_DATE_ERROR = 'VALID'
  AND O_EXP_DATE_ERROR = 'VALID'
  AND DATE_SEQ_ERROR = 'VALID'
  AND O_QTY_ERROR = 'VALID'
```

<divider color="subtle"/>

<row justify="between" align="center">

DEFAULT GROUP – WARNING

<badge color="warning">Warning target</badge>

</row>

No condition required. IDMC automatically routes records that don't match the three explicitly defined groups here.

</box>

Important: Your date flags currently use `UNKNOWN` for NULL values. This design intentionally sends a missing delivered date to WARNING because it is optional, while missing mandatory dates go to REJECT.

3. Add one more validation for delivery performance

Create an Expression port:

`O_LATE_DELIVERY`

```
IIF(
    ISNULL(V_DEL_DATE),
    'N',
    IIF(
        V_DEL_DATE > V_EXP_DATE,
        'Y',
        'N'
    )
)
```

A late delivery will not enter CLEAN. Add this condition to your CLEAN group:

```
AND O_LATE_DELIVERY = 'N'
```

The DEFAULT group will catch late deliveries and send them to WARNING.

4. How to handle duplicates

For duplicates, use `replenishment_id` as your business key.

You need to identify repeated IDs before the Router and populate `O_DUPLICATE_FLAG`.

For example:

replenishment_id	Duplicate flag
REP0000021	N
REP0000022	N
REP0000021	Y
REP0000023	N

You can implement this using an appropriate duplicate-detection step before the Router. Don't simply use `O_REP_ID` trimming as duplicate detection.

---

5. Final mapping structure

<box border radius="lg" padding={3} align="center" gap={2}>
<box background="surface-secondary" radius="md" padding={3} width="100%" align="center">
Source – Replenishment Orders
</box>
<icon name="arrow-down" color="secondary" size="lg"/>
<box background="surface-secondary" radius="md" padding={3} width="100%" align="center">
Expression
<text color="secondary" size="xs">Standardization • Date validation • Quantity validation • Duplicate flag</text>
</box>
<icon name="arrow-down" color="secondary" size="lg"/>
Router
<grid columns={2} gap={2}>
<grid-item>
<box background="#16a34a18" padding={3} radius="md" align="center">
<icon name="check-circle" color="success" size="xl"/>
CLEAN
<text size="xs" color="secondary">Clean target</text>
</box>
</grid-item>
<grid-item>
<box background="#dc262618" padding={3} radius="md" align="center">
<icon name="x-circle" color="danger" size="xl"/>
REJECT
<text size="xs" color="secondary">Error target</text>
</box>
</grid-item>
</grid>
</box>

</box>
</grid-item>
<grid-item>
<box background="#d69e3618" padding={3} radius="md" align="center">
<icon name="alert-triangle" color="warning" size="xl"/>
WARNING
<text size="xs" color="secondary">Default target</text>
</box>
</grid-item>
<grid-item>
<box background="#8b5cf618" padding={3} radius="md" align="center">
<icon name="copy" color="#8b5cf6" size="xl"/>
DUPLICATE
<text size="xs" color="secondary">Duplicate target</text>
</box>
</grid-item>
</grid>
</box>

One final caution: Before running, make sure `O_QTY_ERROR` checks numeric format as well as NULL and negative values. Also, if your IDMC Router evaluates groups independently, keep the duplicate exclusion in the other conditions as shown.

This gives you four clearly separated outputs without losing warning records.

2. Router conditions

Assume you create an `O_DUPLICATE_FLAG` field containing `Y` or `N`.

<box border radius="lg" padding={3} gap={2}>
<row justify="between" align="center">
Group 1 – DUPLICATE
<badge color="discovery">Duplicate target</badge>
</row>

```
  O_DUPLICATE_FLAG = 'Y'
```

<divider color="subtle"/>
<row justify="between" align="center">
Group 2 – REJECT
<badge color="danger">Error target</badge>
</row>

```
  O_DUPLICATE_FLAG = 'N'
  AND
  (
      O_RES_DATE_ERROR <> 'VALID'
      OR O_ALLO_DATE_ERROR <> 'VALID'
      OR O_DIS_DATE_ERROR <> 'VALID'
      OR O_EXP_DATE_ERROR <> 'VALID'
      OR DATE_SEQ_ERROR = 'INVALID'
```

```
      OR O_QTY_ERROR <> 'VALID'
  )
```

Here, `O_EXP_DATE_ERROR` is a new validation port for expected delivery date. Add it to your mapping.

<divider color="subtle"/>

<row justify="between" align="center">

Group 3 – CLEAN

<badge color="success">Clean target</badge>

</row>

```
  O_DUPLICATE_FLAG = 'N'
  AND O_RES_DATE_ERROR = 'VALID'
  AND O_ALLO_DATE_ERROR = 'VALID'
  AND O_DIS_DATE_ERROR = 'VALID'
  AND O_DEL_DATE_ERROR = 'VALID'
  AND O_EXP_DATE_ERROR = 'VALID'
  AND DATE_SEQ_ERROR = 'VALID'
  AND O_QTY_ERROR = 'VALID'
```

<divider color="subtle"/>

<row justify="between" align="center">

DEFAULT GROUP – WARNING

<badge color="warning">Warning target</badge>

</row>

No condition required. IDMC automatically routes records that don't match the three explicitly defined groups here.

</box>

Important: Your date flags currently use `UNKNOWN` for NULL values. This design intentionally sends a missing delivered date to WARNING because it is optional, while missing mandatory dates go to REJECT.

3. Add one more validation for delivery performance

Create an Expression port:

`O_LATE_DELIVERY`

```
IIF(
    ISNULL(V_DEL_DATE),
    'N',
    IIF(
        V_DEL_DATE > V_EXP_DATE,
        'Y',
        'N'
    )
)
```

A late delivery will not enter CLEAN. Add this condition to your CLEAN group:

```
AND O_LATE_DELIVERY = 'N'
```

The DEFAULT group will catch late deliveries and send them to WARNING.

4. How to handle duplicates

For duplicates, use `replenishment_id` as your business key.

You need to identify repeated IDs before the Router and populate `O_DUPLICATE_FLAG`.

For example:

replenishment_id	Duplicate flag
REP0000021	N
REP0000022	N
REP0000021	Y
REP0000023	N

You can implement this using an appropriate duplicate-detection step before the Router. Don't simply use `O_REP_ID` trimming as duplicate detection.

---

5. Final mapping structure

<box border radius="lg" padding={3} align="center" gap={2}>
<box background="surface-secondary" radius="md" padding={3} width="100%" align="center">
Source – Replenishment Orders
</box>
<icon name="arrow-down" color="secondary" size="lg"/>
<box background="surface-secondary" radius="md" padding={3} width="100%" align="center">
Expression
<text color="secondary" size="xs">Standardization • Date validation • Quantity validation • Duplicate flag</text>
</box>
<icon name="arrow-down" color="secondary" size="lg"/>
Router
<grid columns={2} gap={2}>
<grid-item>
<box background="#16a34a18" padding={3} radius="md" align="center">
<icon name="check-circle" color="success" size="xl"/>
CLEAN
<text size="xs" color="secondary">Clean target</text>
</box>
</grid-item>
<grid-item>
<box background="#dc262618" padding={3} radius="md" align="center">
<icon name="x-circle" color="danger" size="xl"/>
REJECT
<text size="xs" color="secondary">Error target</text>
</box>
</grid-item>
</grid>
</box>

</box>
</grid-item>
<grid-item>
<box background="#d69e3618" padding={3} radius="md" align="center">
<icon name="alert-triangle" color="warning" size="xl"/>
WARNING
<text size="xs" color="secondary">Default target</text>
</box>
</grid-item>
<grid-item>
<box background="#8b5cf618" padding={3} radius="md" align="center">
<icon name="copy" color="#8b5cf6" size="xl"/>
DUPLICATE
<text size="xs" color="secondary">Duplicate target</text>
</box>
</grid-item>
</grid>
</box>

One final caution: Before running, make sure `O_QTY_ERROR` checks numeric format as well as NULL and negative values. Also, if your IDMC Router evaluates groups independently, keep the duplicate exclusion in the other conditions as shown.

This gives you four clearly separated outputs without losing warning records.

User

give me the condition for the O*DUPLICATE*FLAG

ChatGPT

For `O_DUPLICATE_FLAG`, use `replenishment_id` as your **primary duplicate identification key**.

However, a normal Expression transformation cannot identify duplicates across multiple rows by itself. You need to use an **Aggregator transformation** before the Router.

Step 1: Aggregator transformation

Group by:

```
replenishment_id
```

Create an output port:

```
COUNT(replenishment_id)
```

Name it:

```
O_ID_COUNT
```

Step 2: Expression transformation

Create `O_DUPLICATE_FLAG`:

```
IIF(O_ID_COUNT > 1, 'Y', 'N')
```

Step 3: Router condition

```
O_DUPLICATE_FLAG = 'Y'
```

Expected output

replenishment*id*	*O*ID*COUNT*	*O*DUPLICATE_FLAG
REP0000021	1	N
REP0000022	2	Y
REP0000022	2	Y
REP0000023	1	N

Important: This identifies all occurrences of a repeated ID as duplicates. If you want to retain the first occurrence and route only subsequent occurrences to DUPLICATE, you need an additional ranking/sequence-based step.

User

GIVE ME ALL 4 CONDITIONS NOW ONE BY ONE

ChatGPT

Use these **4 Router conditions** for your IDMC outputs, one by one.

Assumption: You have already created `O_DUPLICATE_FLAG` and your validation ports in the Expression transformation.

1. DUPLICATE condition

<badge color="warning">DUPLICATE</badge>

```
O_DUPLICATE_FLAG = 'Y'
```

Routes records whose `replenishment_id` appears more than once.

2. REJECT condition

<badge color="danger">REJECT</badge>

```
O_DUPLICATE_FLAG = 'N'
AND
(
    O_RES_DATE_ERROR = 'INVALID'
    OR O_ALLO_DATE_ERROR = 'INVALID'
    OR O_DIS_DATE_ERROR = 'INVALID'
    OR O_DEL_DATE_ERROR = 'INVALID'
    OR DATE_SEQ_ERROR = 'INVALID'
    OR O_QTY_ERROR = 'INVALID'
)
```

Routes records with invalid dates, impossible date sequences, or invalid quantities.

3. CLEAN condition

<badge color="success">CLEAN</badge>

```
O_DUPLICATE_FLAG = 'N'
AND O_RES_DATE_ERROR = 'VALID'
```

```
AND O_ALLO_DATE_ERROR = 'VALID'
AND O_DIS_DATE_ERROR = 'VALID'
AND O_DEL_DATE_ERROR = 'VALID'
AND DATE_SEQ_ERROR = 'VALID'
AND O_QTY_ERROR = 'VALID'
```

Routes records that pass all validations.

4. WARNING condition

<badge color="info">WARNING — Default group</badge>

Do not create a condition for this group. Keep the Router's Default Group connected to your WARNING target.

It will receive records that are neither DUPLICATE, REJECT, nor CLEAN—for example, records with optional missing values.

Important: Create the groups in this order: `DUPLICATE`, `REJECT`, `CLEAN`, and connect the Default Group to `WARNING`.

These conditions assume your validation flags return exactly `VALID` or `INVALID`, and your duplicate flag returns `Y` or `N`.

User

See I have 4 date error res*date*error, allo*date*error, dis*date*error,*del*date*error it works in a logic of iif(isnull(respective*date_column),'unknown','valid')

for quantity
if the data is null or less than 0 then it is marked as invalid else valid

ChatGPT

Got it. Your validation flags work like this:

- Date error flags: `UNKNOWN` when the date is NULL; otherwise `VALID`.
- Quantity error flag: `INVALID` when any quantity is NULL or less than 0; otherwise `VALID`.

So, we need to update the four Router conditions to match your actual flag values.

1. DUPLICATE condition

```
O_DUPLICATE_FLAG = 'Y'
```

2. REJECT condition

```
O_DUPLICATE_FLAG = 'N'
AND
(
    O_DIS_DATE_ERROR = 'UNKNOWN'
    OR DATE_SEQ_ERROR = 'INVALID'
    OR O_QTY_ERROR = 'INVALID'
)
```

3. CLEAN condition

```
O_DUPLICATE_FLAG = 'N'
AND O_RES_DATE_ERROR = 'VALID'
AND O_ALLO_DATE_ERROR = 'VALID'
AND O_DIS_DATE_ERROR = 'VALID'
AND O_DEL_DATE_ERROR = 'VALID'
AND DATE_SEQ_ERROR = 'VALID'
AND O_QTY_ERROR = 'VALID'
```

4. WARNING condition

Keep the Router's **Default Group** connected to WARNING.

This will route records that have optional missing dates, such as missing `delivered_date` or `allocated_date`, provided they don't meet the REJECT condition.

Important: I have treated `request_date` and `dispatch_date` as mandatory, and `allocated_date` and `delivered_date` as optional. If your business rules define different mandatory dates, we should adjust the REJECT condition accordingly. Also, make sure `DATE_SEQ_ERROR` is actually created and returns `VALID` or `INVALID`.

If you want, I can:

- Define the DUPLICATE condition for Router
- Define the REJECT condition for Router
- Define the CLEAN condition for Router